classmate
Date
Page 1

॥ ॐ नमो भगवते वासुदेवाय नमः ॥

1 June 2026 – 6:23 PM – Monday

ML Course – Andrew Ng – Lecture-1

✡ Pre Requisite

- Probability
- Random variable
- Expected value of a random variable
- Variance of a random variable
- Linear Algebra
- Matrix
- Vector
- Multiplication of two matrices
- Eigen Vector
- Convex optimization algorithms
- HMM
- Time Series

→ Choose narrow task in which computers & AI outperforms humans like speech recognition on a certain type of task, we can maybe surpass human level performance.

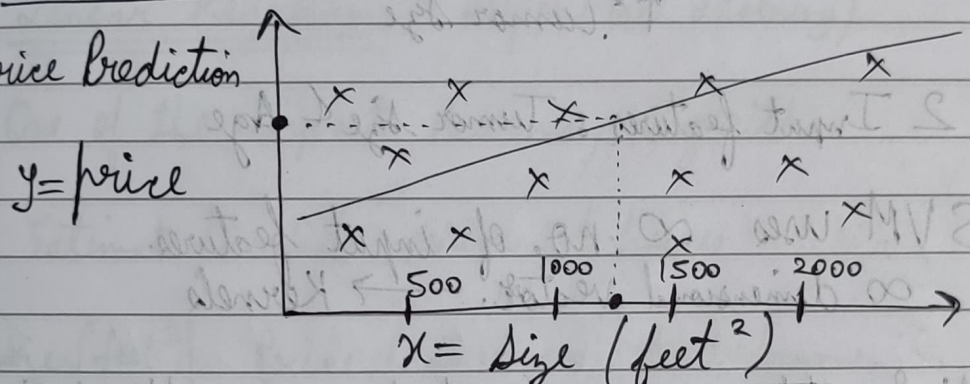
→ Gives computers the ability learn without being explicitly programmed.

→ Arthur Samuel (1959). ML : Field of study that gives computers the ability to learn without being explicitly programmed.

→ Tom Mitchell (1998) Well Posed Learning Problem: A computer program is said to learn from experience E with respect to some task T and some performance measure P, if its performance on T, as measured by P, improves with experience E.

* Supervised Learning

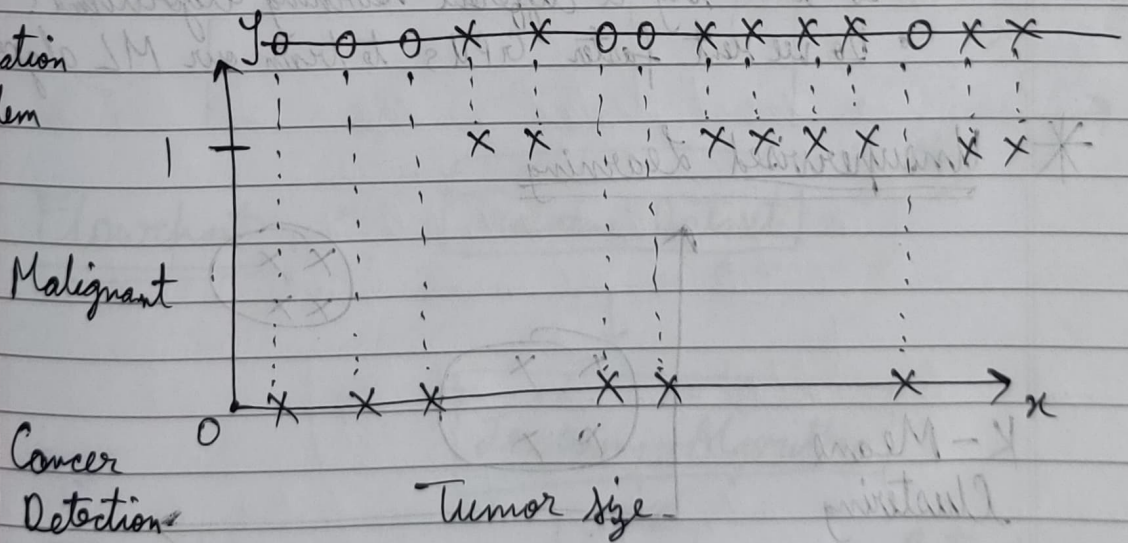
House Price Prediction



Regression problem

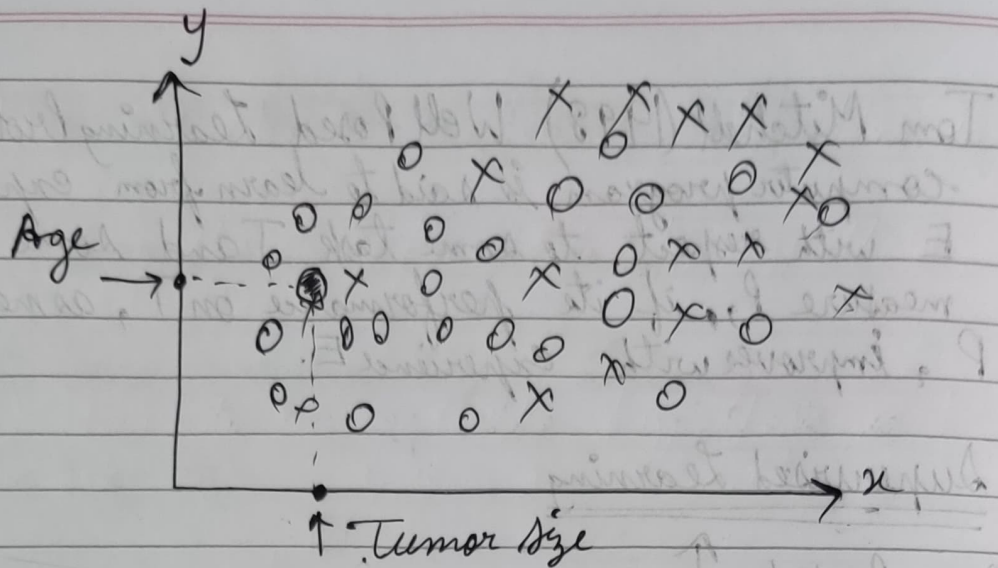
Value y we are trying to predict is continuous

Classification Problem



Cancer Detection

Output is discrete : 0 or 1



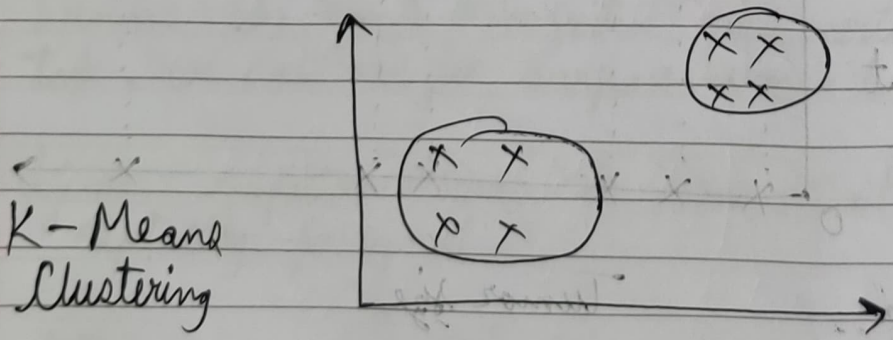
2 Input features : Tumor size & Age

→ SVM uses ∞ no. of input features.
∞ dimensional vector. → Kernels

→ ML Practitioners are very strategic i.e. the skill at deciding. Lot of decisions to make.

- Do we collect more data?
- Do we try a different learning algorithms?
- Do we rent faster GPUs to train our ML algorithm?

* Unsupervised Learning



only X is given no Y
Unlabeled data

→ Cocktail Party Problem — ICA
Independent Component Analysis

→ Learning Analogies

2 June 2026 — Tuesday — 8:49 AM

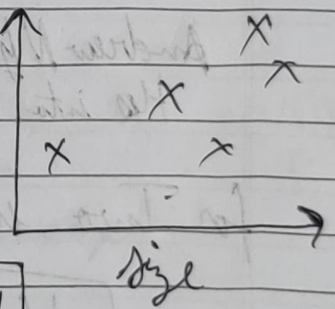
* Linear Regression (Supervised Learning)

One of the simplest learning algorithm

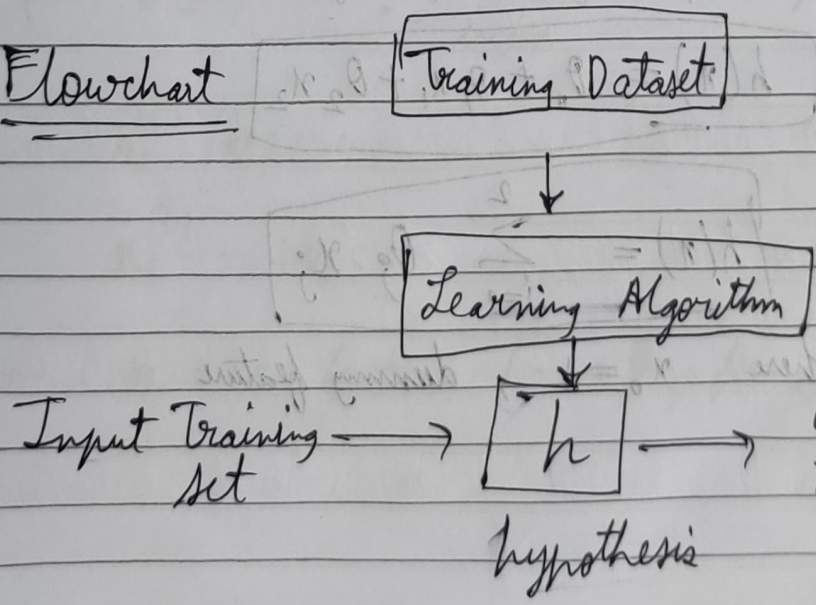
→ Estimate / Predict prices of houses

Data from Portland Oregon

Size (feet²)	Price ($1000s)	~~#~~ bedrooms
2104	400	
1416	232	
1534	315	
852	178	



* Flowchart



✡ Way we structure a learning algorithm is important

✡ Designing the workflow, data set, hypothesis, how to represent the hypothesis

→ ① How to represent the hypothesis is 1st question we ask when designing an algorithm.

H?

$$h(x) = \theta_0 + \theta_1 x$$

Linear fn & technically it is affine fn

One input feature x, size (feet^2)

Andrew Ng father in law lives outside Portland Has into Real Estate

for Two input feature, size & # bedrooms

$$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2$$

$$h(x) = \sum_{j=0}^{2} \theta_j x_j$$

where $x_0 = 1$, dummy feature

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix} \qquad x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \end{bmatrix}$$

θ = Parameters

→ Job of Algo is to choose parameters θ, that allows you to make good predictions about your prices of house

m = # training examples
(# rows in table above)

X = input / features.

y = output / target variable

(x, y) = training example

$(x^{(i)}, y^{(i)})$ = i^{th} training example
i = index

$x_1^{(1)} = 2104$

$x_1^{(2)} = 1416$

n = # features ($= 2$, here)

राधा

classmate
Date
Page 7

$$h(x) = \sum_{j=0}^{n} \theta_j x_j$$

→ Choose θ st $h(x) \approx y$ for training examples

$$h_\theta(x) = h(x)$$

hypothesis depends on both θ & x

→ Linear Regression algorithm also called ordinary least squares (OLS)

$$\text{minimize}_{\theta} \sum_{i=1}^{m} \left[h_\theta(x^{(i)}) - y \right]^2$$

$$\underset{\theta}{\text{minimize}} \; \frac{1}{2} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

→ $J(\theta)$ minimize $J(\theta)$ Cost fn

* Generalized linear models

Squarred error corresponds to Gaussian

finding value of θ which minimizes $J(\theta)$

मृत्युंजय महादेव

* Gradient Descent

Start with some θ (say $\theta = \vec{0}$)

vector of 0s (zeros)

Keep changing θ to reduce $J(\theta)$

Trying to go lowest possible elevation

Which direction to go in 360° in order to reduce $J(\theta)$

Steepest gradient downhill. Until we get to local optimum.

Multiple Local minima are possible.

$$\theta_j := \theta_j - \alpha \cdot \frac{\partial}{\partial \theta_j} J(\theta)$$

α = learning rate, $j = 0, 1, 2$

Derivate of a fn defines the direction of steepest descent

Try with 0.01 for α & then ↑ or ↓ as it goes

राधा

$$\frac{\partial}{\partial \theta_j} J(\theta) = \frac{\partial}{\partial \theta_j} \frac{1}{2} \left(h_\theta(x) - y \right)^2$$

$$= 2 \cdot \frac{1}{2} \left(h_\theta(x) - y \right) \cdot \frac{\partial}{\partial \theta_j} \left(h_\theta(x) - y \right)$$

$$= \left(h_\theta(x) - y \right) \cdot \frac{\partial}{\partial \theta_j} \left(\theta_0 x_0 + \theta_1 x_1 + \ldots\ldots + \theta_n x_n \right)$$

→ Partial derivate of each of these terms with respect to θ_j is going to be 0 except for the term j corresponding

$$= \boxed{\left(h_\theta(x) - y \right) \cdot x_j}$$

$$\frac{\partial}{\partial \theta_j} \left(\theta_0 x_0 + \theta_1 x_1 + \ldots\ldots + \theta_n x_n - y \right)$$

$$j \in [0, n]$$

so here the term will be

$$= \frac{\partial}{\partial \theta_j} \left(\theta_j x_j \quad - y \right)$$

$$= \boxed{x_j}$$

$$\frac{\partial}{\partial \theta_j} J(\theta) = \left[h_\theta(x) - y\right] \cdot x_j$$

~~$\theta_j := \theta_j - \alpha \cdot [h_\theta(x) - y] \cdot x_j$~~

-y) $$\theta_j := \theta_j - \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right) \cdot x_j^{(i)}$$

m = # training examples

$x^{(i)}$ = i^{th} training example input feature

$y^{(i)}$ = target label in i^{th} training example

$$\frac{\partial}{\partial \theta_j} J(\theta) = \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right) \cdot x_j^{(i)}$$

→ Gradient descent algorithm is to repeat until convergence and in each iteration of gradient descent we do this update (:=) for $j = 0, 1, 2, \ldots, n$

n = # features (here $n = 2$)

→ When we plot cost fn of θ for linear regression model. $J(\theta)$ is a quadratic fn.

$J(\theta)$ does not have local optima it only has global optima

राधा

classmate
Date
Page 11

→ Contours of the fn, the direction of steepest descent is always at 90 degree is always orthogonal to the contour direction.

→ Only 1 global minimum

→ If α is too large it can overshoot & if α is too low algo will be slow & need more iterations
In practice we try a few values & then see what value of the learning rate allows us to most efficiently.

If $J(\theta) \uparrow \Longrightarrow \alpha$ is $\uparrow$

→ Try multiple values of α & usually try them on an exponential scale

✡ Contour

a fn of 2 variables

$$J(w, b)$$

A contour line is a curve where the fn value stays constant

for eg $J(w, b) = 10$

$$\nabla J = \left[\frac{\partial J}{\partial w}, \frac{\partial J}{\partial b} \right]$$

Gradient points in the direction of max ↑ of fn

So:

$+\nabla J =$ ↑ direction (uphill)

$-\nabla J =$ ↓ direction (downhill)

$$\theta := \theta - \alpha \nabla J$$

which means move opposite to the gradient.

$\nabla J \cdot t = 0$] dot product zero @ 90° as $\cos 90° = 0$

→ Suppose we are standing ~~at~~ on a contour. Moving along that contour does not change J

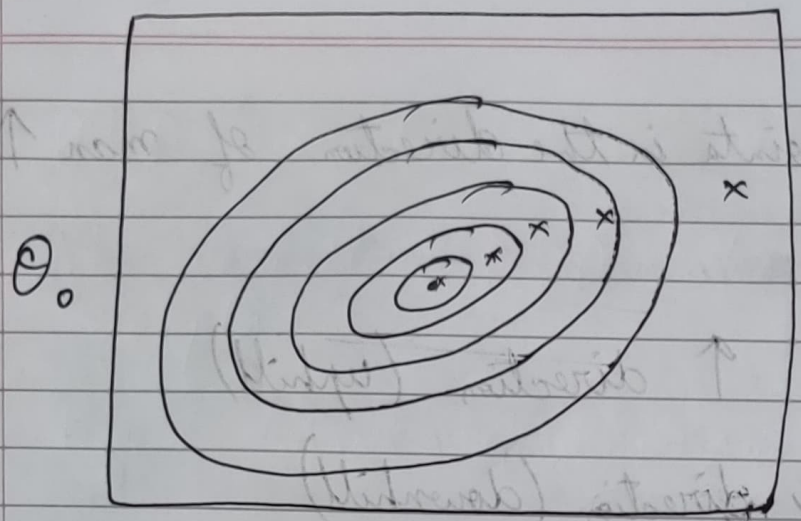
So the rate of Δ of J in the contour direction must be zero

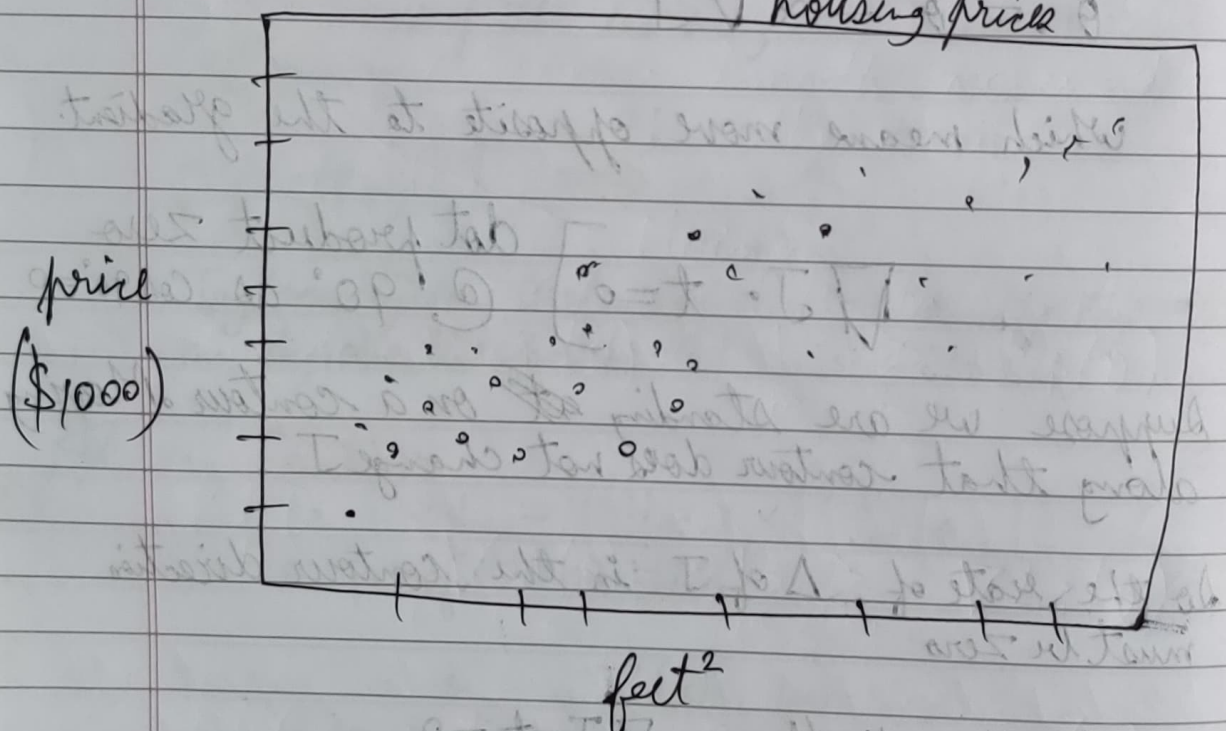
Mathematically $\nabla J \cdot t = 0$

t = tangent vector to contour
∇J = gradient

dot product of zero means @ 90°

Ellipses are contours





$m = 49$ # parameters

→ Another name is Batch gradient descent it goes to global minimum & stopped

→ Repeat {

for j = 1 to m {

$$\theta_j := \theta_j - \alpha \left(h_\theta(x^{(i)}) - y^{(i)} \right) x_j^{(i)}$$

}

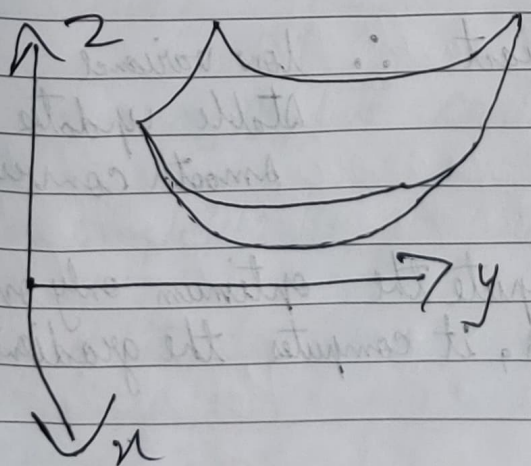
}

→ Stochastic Gradient Descent

As we run SGD will never quite converge. When we have very large datasets SGD allows our implementation i.e. algorithm to run faster. As very large dataset SGD is used much more in practice than BGD.

→ BGD converges smoothly because it computes the exact gradient of the entire dataset.

The Cost fn looks like a bowl where the bottom is the minimum cost

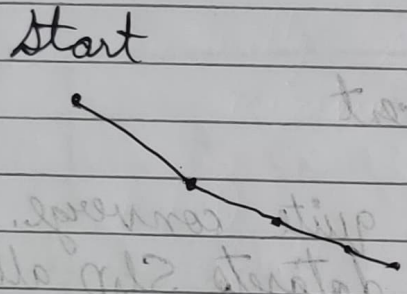


→ **BgD**

At every step, BgD looks at all training examples.

: For a dataset of 10,000 samples

Step 1, 2, 3 → Uses all 10,000 samples computes exact gradient



Smooth movement towards the minimum

→ Cost fn $J(\theta) = \frac{1}{m} \sum_{i=1}^{m} L_i(\theta)$

BgD $\nabla J(\theta) = \frac{1}{m} \sum_{i=1}^{m} \nabla L_i(\theta)$

BgD computes: $\theta = \theta - \alpha \cdot \nabla J(\theta)$

- Gradient is exact ∴ low variance
  Stable updates
  Smooth convergence
- BgD does not compute the optimum only once. Instead at every iteration, it computes the gradient using the entire dataset

Then each update is

$$\theta_{t+1} = \theta_t - \alpha \frac{1}{m} \sum_{i=1}^{m} \nabla L_i(\theta_t)$$

→ **SGD**

Instead of all examples:

Pick one example i

Gradient estimate:

$$g_i = \nabla L_i(\theta)$$

Update: $\theta = \theta - \alpha g_i$

Problem: ~~$g_i \neq \nabla J(\theta)$~~ $g_i \neq \nabla J(\theta)$ for most samples

Each example pulls the parameter in a slightly different direction

Thus: $g_i = \nabla J(\theta) + \text{noise}$

So every update contains noise.

pick one sample i: $\theta_{t+1} = \theta_t - \alpha \nabla L_i(\theta_t)$

Now update depends on which sample was chosen
Sample 1 → update
Sample 2 → update

→ Why doesn't SGD converge exactly?

∴ Imagine true minimum is here : x

1) In BGD

Gradient becomes $\nabla J \approx 0$

So update is $\Delta\theta \approx 0$

eventually stops at almost exactly at minimum.

2) In SGD

Even at minimum : Different samples still produce different gradients

S1 $\Delta = +0.2$
S2 $\Delta = -0.1$
S3 $\Delta = +0.15$
S4 $\Delta = -0.25$

Average $= 0$

Individually $\neq 0$

SGD keeps moving around the minimum every sample pulls the parameter in a different direc"

It oscillates forever

Thus

True gradient + Noise

Hence Zig - Zagging

→ Why use SGD then?

Because BGD becomes expensive

Suppose: 100M samples

BGD computes gradients on 100M samples before every update.

Very Slow

SGD: 1 sample → update
1 sample → update
1 sample → update

Much faster

→ Mini-Batch Gradient Descent

Modern DL uses

Batch GD: all samples
SGD: 1 sample
Mini BGD: 32, 64, 128, 256 samples

Mini Batch is the compromise

Path: smooth enough & fast enough

This is why virtually all modern neural networks use mini-batch gradient descent rather than pure SGD or pure BGD.

→ Why BGD converges ?

Suppose $J(\theta)$ is convex cost fn ∪

@ θ_1,

$\nabla J(\theta)$ points towards steepest ↑

$-\nabla J(\theta)$ points towards steepest ↓

Update rule

$$\theta_{t+1} = \theta_t - \alpha \cdot \nabla J(\theta_t)$$

for sufficiently small α,

$$J(\theta_{t+1}) < J(\theta_t)$$

every iteration

Cost continuously ↓

100, 50, 20, 10, 5, 2, 1, 0.5

→ Why SGD oscillates?

$$g_i = \nabla L_i(\theta)$$

For a random sample i

$$E[g_i] = \nabla J(\theta)$$

SGD gradient is an unbiased estimator of true gradient

$$Var(g_i) > 0$$

So $g_i = \nabla J(\theta) + \varepsilon_i$

where $E[\varepsilon_i] = 0$

but $Var(\varepsilon_i) \neq 0$

Near the minimum.

$$\nabla J(\theta) \approx 0$$

$$\therefore g_i \approx \varepsilon_i$$

Noise dominates. Update never actually become zero

* Normal Equation

Matrix derivate notation that allows us to drive the normal eq^n in roughly 4 lines of LA

$\nabla_\theta J(\theta)$ derivate of $J(\theta)$ wrt θ

$\theta \in \mathbb{R}^{n+1}$ θ is 3D vector

here $n = 2$
features

$$\nabla_\theta J(\theta) = \begin{bmatrix} \frac{\partial J}{\partial \theta_0} \\ \frac{\partial J}{\partial \theta_1} \\ \frac{\partial J}{\partial \theta_2} \end{bmatrix}$$

$A \in \mathbb{R}^{2\times 2}$ 2 × 2 matrix

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$$

$$f(A) = A_{11} + A_{12}^2$$

$f : \mathbb{R}^{2\times 2} \longrightarrow \mathbb{R}$

$$f\left(\begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}\right) = 5+6^2$$

$$\nabla_A f(A) = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \frac{\partial f}{\partial A_{12}} \\ \frac{\partial f}{\partial A_{21}} & \frac{\partial f}{\partial A_{22}} \end{bmatrix}$$

$$\nabla_A f(A) = \begin{bmatrix} 1 & 2A_{12} \\ 0 & 0 \end{bmatrix}$$

as $f(A) = A_{11} + A_{12}^2$

To derive normal equation of $J(\theta)$ Compute derivate of $J(\theta)$ wrt θ and set it to vector 0

$$\nabla_\theta J(\theta) \overset{\text{set}}{=} \vec{0}$$

and solve for θ mining & maxima

If A is square matrix $(A \in \mathbb{R}^{2\times 2})$

$tr\,A$ = sum of diagnol entries = $\sum_i A_{ii}$

$$tr\,A = tr\,A^T$$

→ $f(A) = \text{tr}\, AB$

B is fixed matrix

- $\nabla_A f(A) = B^T$ ⟶ Prove yourself
- $\text{tr}\, AB = \text{tr}\, BA$
- $\text{tr}\, ABC = \text{tr}\, CBAB$
- $\nabla_A \text{tr}\, AA^T C = CA + C^T A$

analogous ↳ $\frac{d}{da} a^2 c = 2ac$

$$J(\theta) = \frac{1}{2} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$

$$X\theta = \begin{bmatrix} — (x^{(1)})^T — \\ — (x^{(2)})^T — \\ \vdots \\ — (x^{(m)})^T — \end{bmatrix} \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \end{bmatrix}$$

$$X\theta = \begin{bmatrix} (x^{(1)})^T \theta \\ (x^{(2)})^T \theta \\ \vdots \\ (x^{(m)})^T \theta \end{bmatrix} = \begin{bmatrix} h_\theta(x^{(1)}) \\ h_\theta(x^{(2)}) \\ \vdots \\ h_\theta(x^{(m)}) \end{bmatrix}$$

$$\vec{y} = \begin{bmatrix} y^{(1)} \\ y^{(2)} \\ \vdots \\ y^{(m)} \end{bmatrix}$$

$$J(\theta) = \frac{1}{2} (X\theta - y)^T (X\theta - y)$$

$$X\theta - y = \begin{bmatrix} h_\theta(x^{(1)}) - y^{(1)} \\ h_\theta(x^{(2)}) - y^{(2)} \\ \vdots \\ h_\theta(x^{(m)}) - y^{(m)} \end{bmatrix}$$

$$z^T \cdot z = \sum_i z^2$$

A vector transpose itself is the sum of squares of the elements

$$(X\theta - y)^T \cdot (X\theta - y) = (X\theta - y)^2$$

$$J(\theta) = \frac{1}{2} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - (y^{(i)})\right)^2$$

$$J(\theta) = \frac{1}{2} (X\theta - y)^T \cdot (X\theta - y)$$

$$J(\theta) = (X\theta - y)^2$$

$$\nabla_\theta J(\theta) = \nabla_\theta \frac{1}{2} (X\theta - y)^T (X\theta - y)$$

$$= \frac{1}{2} \nabla_\theta (\theta^T X^T - y^T)(X\theta - y)$$

$$= \frac{1}{2} \nabla_\theta \left[\theta^T X^T X\theta - \theta^T X^T y - y^T X\theta + y^T y\right]$$

Lecture notes

$$= \frac{1}{2} \left[X^T X\theta + X^T X\theta - X^T y - X^T y\right]$$

$$= X^T X\theta - X^T y \overset{set}{=} \vec{0}$$

$$X^T X\theta = X^T y \quad \text{"Normal Eq"}$$

$$\therefore \theta = (X^T X)^{-1} X^T y$$

4 June 2026
Thursday
7:00 PM

Proof

$$\nabla_A f(A) = B^T$$

where $f(A) = tr(AB)$
B is fixed matrix

$$A \to A + dA$$

then $$df = tr((dA)B)$$

as we know $tr(AB) = tr(BA)$

so $$df = tr(B(dA))$$

$$\boxed{tr(ABC) = tr(BCA) = tr(CAB)}$$

$$\boxed{tr(X) = tr(X^T)}$$

$$tr(X^T y) = tr((X^T y)^T) = tr(y^T X)$$

$$\boxed{tr(X^T y) = tr(Y^T X)}$$

$$tr(X^T Y) = \sum_{i,j} X_{ij} Y_{ij}$$

$$\nabla y = \frac{\partial f}{\partial A} = \begin{bmatrix} \frac{\partial f}{\partial A_{11}} & \frac{\partial f}{\partial A_{12}} \\ \frac{\partial f}{\partial A_{21}} & \frac{\partial f}{\partial A_{22}} \end{bmatrix}$$

$$\therefore df = \sum_{ij} y_{ij}\, dA_{ij}$$

$$\therefore df = tr(y^T\, dA)$$

$$y = \frac{\partial f}{\partial A}$$

$$\therefore df = tr\left(\left(\frac{\partial f}{\partial A}\right)^T dA\right)$$

We get $df = tr(B\ dA)$

$$\therefore \left(\frac{\partial f}{\partial A}\right)^T = B$$

$$\frac{\partial f}{\partial A} = B^T$$

$$\boxed{\nabla_A f(A) = B^T}$$

Hence proved.

$$\boxed{\frac{\partial}{\partial A} tr(AB) = B^T}$$

Proof

$$\nabla_A \, tr\, AA^T C = -CA + C^T A$$

$$(ZZ^T)^T = ZZ^T$$

and $$(Z^T Z)^T = Z^T Z$$

$$tr(ZZ^T) = \sum_{i,j} Z_{ij}^2$$

$$tr(ZZ^T) = \|Z\|_F^2$$

$\|Z\|_F$ is Frobenius Norm

$$Z^T \cdot Z = \sum_i Z^2$$

$$\|Z\|_F = \sqrt{\sum_{ij} Z_{ij}^2}$$

$$\longrightarrow \quad tr(AA^T C) = tr(CAA^T)$$

$$\frac{\partial}{\partial A} tr(AA^T C) = \frac{\partial}{\partial A} tr(CAA^T)$$

$$d(CAA^T) = d(CA)\cdot A^T + d(A^T)\cdot CA$$

$$= ACA^T(dA) + CA(dA^T)$$

$$= CA^T(dA)A^T + CA(dA)^T$$

$$f(A) = tr(AA^T C)$$

TP $\frac{\partial f}{\partial A} = CA + C^T A$

$$tr(AA^T C) = tr(CAA^T)$$

$$f(A) = tr(CAA^T)$$

$$df = d\, tr(CAA^T)$$

$$df = tr(d(CAA^T))$$

Now $d(CAA^T) = C(dA)A^T + CA(dA)^T$

$$df = tr(C(dA)A^T + CA(dA)^T)$$

$$df = tr(C(dA)A^T) + tr(CA(dA)^T)$$

$$df = tr(A^T C(dA)) + tr((CA(dA)^T)^T)$$

$$df = tr(A^T C(dA)) + tr((dA)A^T C^T)$$

$$df = tr(A^T C(dA)) + tr(A^T C^T (dA))$$

$$df = tr((A^T C + A^T C^T)\, dA)$$

Now compare $df = tr\left(\left(\frac{\partial f}{\partial A}\right)^T dA\right)$

$$\left(\frac{\partial f}{\partial A}\right)^T = A^T C + A^T C^T$$

$$\frac{\partial f}{\partial A} = \left(A^T C + A^T C^T\right)^T$$

$$\frac{\partial f}{\partial A} = \cancel{(C + C^T)A} \quad C^T A + CA$$

Hence proved

$$\boxed{\frac{\partial f}{\partial A} = CA + C^T A}$$

Similarly $\frac{\partial}{\partial A} tr(A^T A) = 2A$

as $tr(A^T A) = ||A||_F^2$

$$\frac{\partial}{\partial A} ||A||_F^2 = 2A$$

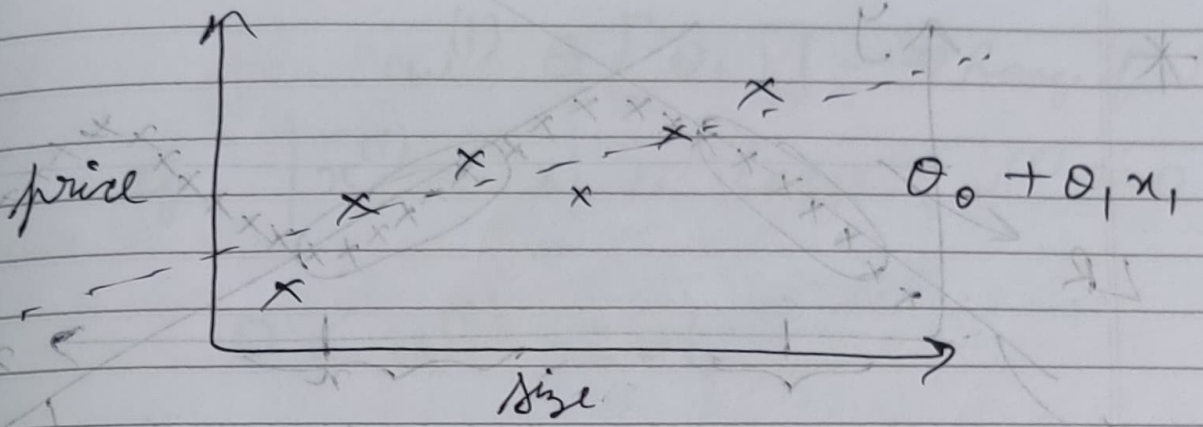
* **Lecture -3** Locally weighted & Logistic Regression

Outline

- Linear Regression
- Locally weighted Regression
- Probabilistic interpretation
- Logistic Regression
- Newtons Method

* **Recap**

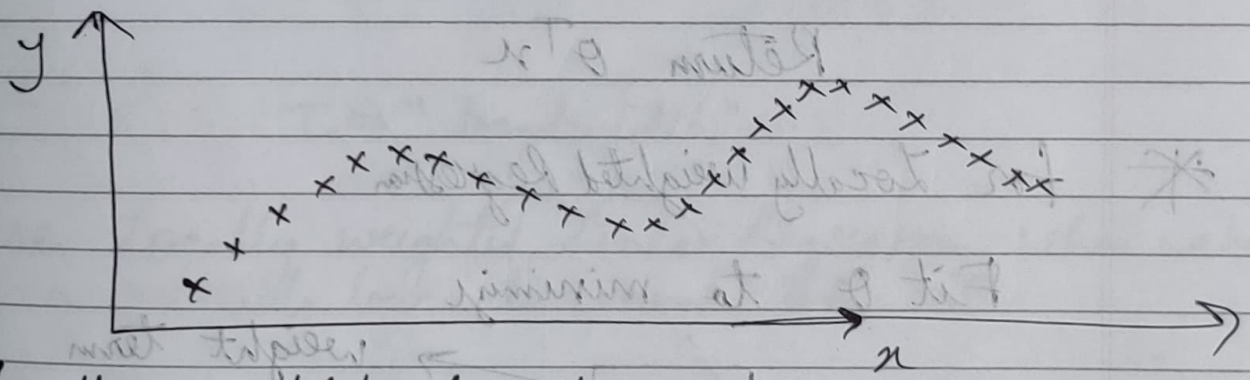
- $(x^{(i)}, y^{(i)})$ ⟶ i^{th} example
- $x^{(i)} \in \mathbb{R}^{n+1}$, $y^{(i)} \in \mathbb{R}$, $x_0 = 1$
- m = # examples, n = # features
- $$h_\theta(x) = \sum_{j=0}^{m} \theta_j x_j = \theta^T x$$
- $$J(\theta) = \frac{1}{2} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)} \right)^2$$



- Do we want to fit straight line or some another fn

$$\theta_0 + \theta_1 x_1 + \theta_2 x^2 \quad (x_1, x_2)$$

or $\theta_0 + \theta_1 x + \theta_2 \sqrt{x} + \theta_3 \log(x)$

- 

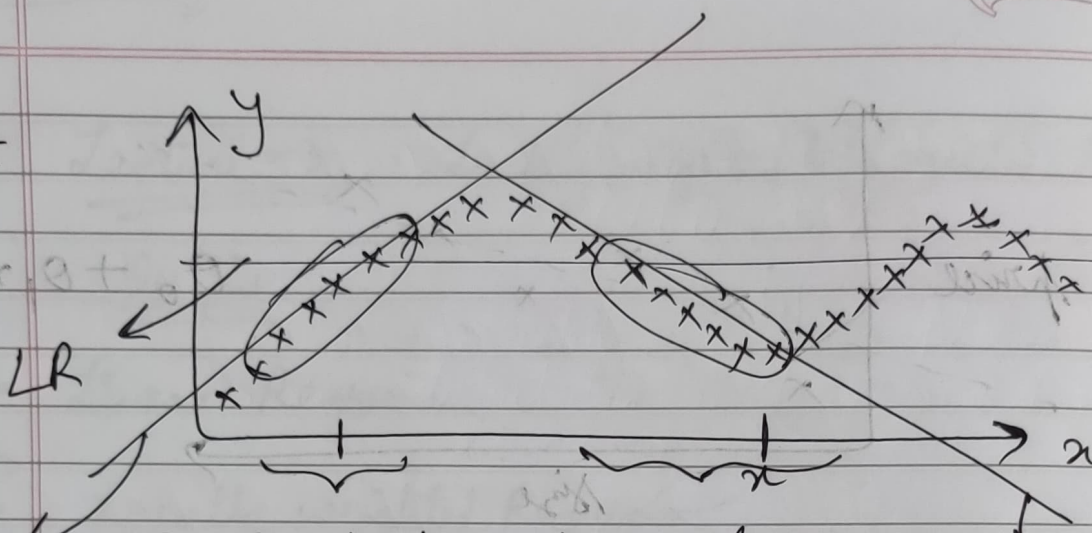
* Locally weighted algorithms.

→ "Parametric" & "Non Parametric" Learning algorithms

Fit fixed set of parameters (θ_i) to data

→ "Non Parametric" learning algorithms

Amount of data/parameters we need to keep grows (linearly) with size of data.



To estimate h at certain x.

Linear Regression: Fit θ to minimize

Cost fn $\frac{1}{2} \sum_i \left(y^{(i)} - \theta^T x^{(i)} \right)^2$

Return $\theta^T x$

* for Locally weighted Regression

Fit θ to minimize

$$\sum_{i=1}^{m} \boxed{w^{(i)}} \left(y^{(i)} - \theta^T x^{(i)} \right)^2$$

weight term

where $w^{(i)}$ is a "weight fn"

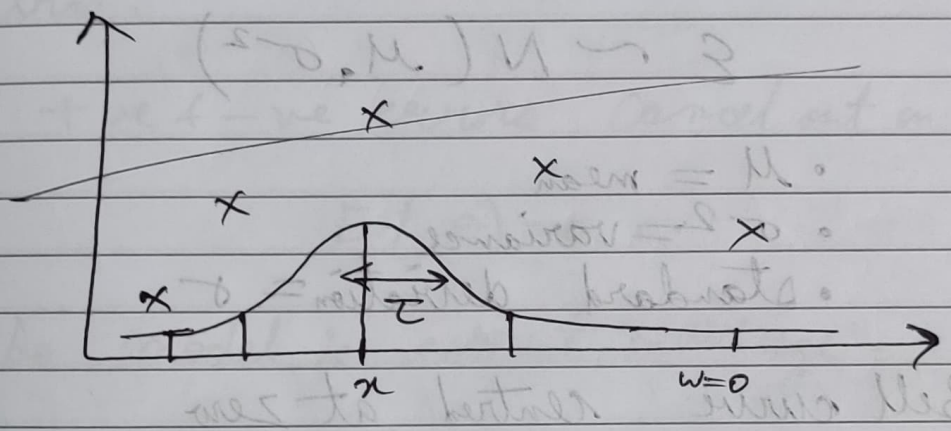
default choice $w^{(i)} = \exp\left(- \frac{(x^{(i)} - x)^2}{2 \cdot \tau^2} \right)$

if $|x^{(i)} - x|$ is small ; $w^{(i)} \approx 1$.

$w^{(i)} \in [0, 1] \longrightarrow$ Range of $w^{(i)}$

if $|x^{(i)} - x|$ is large ; $w^{(i)} \approx 0$

$0 \leq w^{(i)} \leq 1$



τ = " bandwidth "

- Use Locally weighted Linear Regression when we have a relatively low dimensional data set

features ↓ n ↓

* Why Least Squares ?

Assume $y^{(i)} = \theta^T x^{(i)} + \varepsilon^{(i)}$

↓ error, random noise unmodelled effects

$\varepsilon^{(i)} \sim N(0, \sigma^2)$

$P(\varepsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\sigma} \exp\left(\frac{-(\varepsilon^{(i)})^2}{2\sigma^2}\right)$

$$\varepsilon^{(i)} \sim N(0, \sigma^2)$$

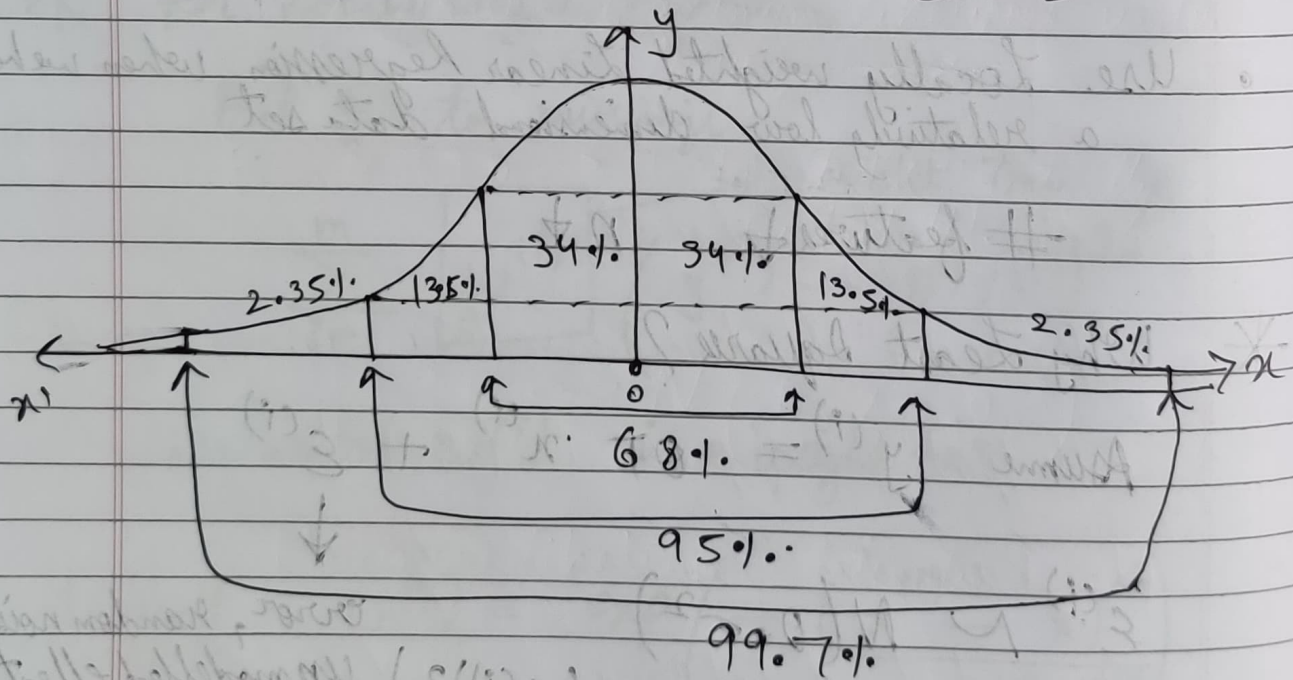
→ The random variable ε follows a Normal i.e. Gaussian distribution with mean 0 & variance σ^2.

$$\varepsilon \sim N(\mu, \sigma^2)$$

- μ = mean
- σ^2 = variance
- Standard deviation = σ.

→ Bell curve centred at zero

$$Var(X) = E(X^2) - [E(X)]^2$$



$$y = \theta^T x + \varepsilon \quad \text{or}$$
$$y = h_\theta(x) + \varepsilon$$

- $h_\theta(x)$ is true relationship
- ε is random noise

- Why mean=0?

Because we assume the noise has no systematic bias.

+ve & -ve errors cancel out on average

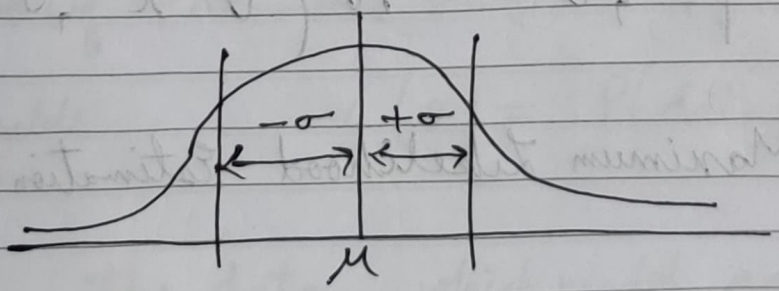
$$E[\varepsilon] = 0$$

So model is correct "on average".

$$\varepsilon^{(i)} \sim N(0, \sigma^2)$$

$$P(\varepsilon^{(i)}) = \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(\varepsilon^{(i)})^2}{2\sigma^2}\right)$$

→ Probability density fn
It integrates to 1. Gaussian density



σ Controls width of the bell curve

Likelihood → "Paramaterized by"

$$P(y|x;\theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y-\theta^T x)^2}{2\sigma^2}\right)$$

Maximising likelihood ~~≠~~ minimizes $\varepsilon^{(i)}$

$$\sum_i (y_i - \theta^T x_i)^2$$

Least Squares Cost fn

$$J(\theta) = \frac{1}{2m} \sum_i (h_\theta(x_i) - y_i)^2$$

Least Squares = Maximum Likelihood under Gaussian noise.

- ε is IID → Independent & Identically Distributed.

$$y^{(i)} | x^{(i)}; \theta \sim N(\theta^T x^{(i)}, \sigma^2)$$

Maximum Likelihood : Estimation

$$L(\theta) = P(\vec{y} \mid x; \theta)$$

$L(\theta)$ = Likelihood of θ

$$L(\theta) = \prod_{i=1}^{m} P(y^{(i)} \mid x^{(i)}; \theta)$$

$$L(\theta) = \prod_{i=1}^{m} \frac{1}{\sqrt{2\pi}\,\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

Likelihood of Parameters

Probability of Data

→ Probability of Data : $P(\text{Data} \mid \text{Model})$

asks given the model, what data might occur?

θ fixed
y variable

$$P(y \mid x; \theta)$$

→ Likelyhood of Parameters : $L(\theta) = P(y \mid x; \theta)$

θ variable
y fixed

$$L(\theta) = P(\text{data} \mid \theta)$$

Given the data, which model parameter is most plausible?

Log is strictly monotonically ↑ fn

* Log Likelihood

$$l(\theta) = \log L(\theta)$$

$$= \log \prod_{i=1}^{m} \frac{1}{\sqrt{2\pi}\sigma} \cdot \exp(\dots)$$

$$= \sum_{i=1}^{m} \left[\log \frac{1}{\sqrt{2\pi}\sigma} + \log \exp(\dots) \right]$$

Constant term ←

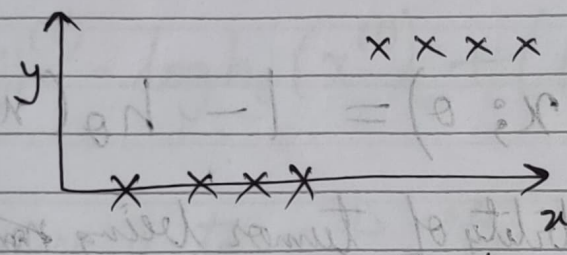
$$= \boxed{m \log \frac{1}{\sqrt{2\pi\sigma^2}}} + \sum_{i=1}^{m} -\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}$$

→ MLE : Maximum Likelihood Estimation

Choose θ to ↑ $L(\theta)$

* Classification

$y \in \{0, 1\}$ Binary classification



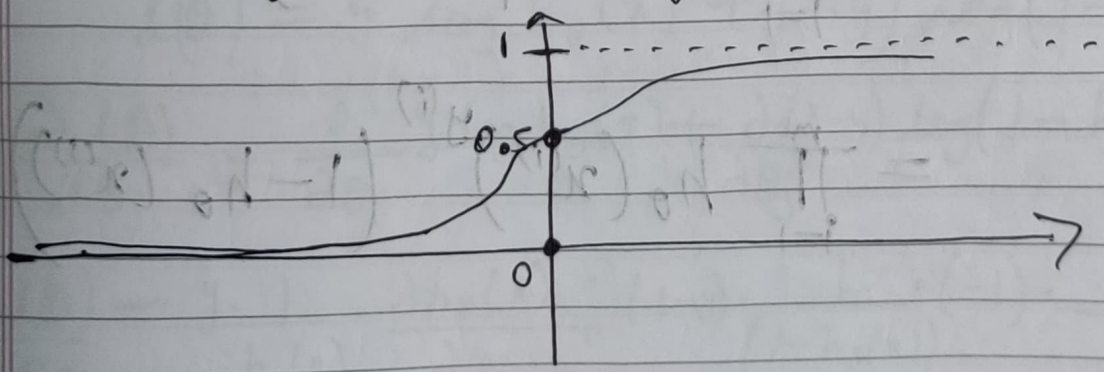
Logistic Regression for classification problem

We want $h_\theta(x) \in [0, 1]$ → Range

$$h_\theta(x) = g(\theta^T x) = \frac{1}{1 + e^{-\theta^T x}}$$

$$g(z) = \frac{1}{1 + e^{-z}}$$

Sigmoid or logistic fn



$$P(y=1 \mid x;\theta) = h_\theta(x)$$

Probability of tumor being malignant (1)

$$P(y=0 \mid x;\theta) = 1 - h_\theta(x)$$

Probability of tumor being ~~non malignant~~ benign (0)

$$y \in \{0,1\}$$

One compressed eqⁿ

$$\rightarrow P(y \mid x;\theta) = h_\theta(x)^y \left(1 - h_\theta(x)\right)^{1-y}$$

Likelihood of parameters

$$L(\theta) = P(\vec{y} \mid x;\theta)$$

$$= \prod_{i=1}^{m} P(y^{(i)} \mid x^{(i)};\theta)$$

$$= \prod_{i=1}^{m} h_\theta(x^{(i)})^{y^{(i)}} \left(1 - h_\theta(x^{(i)})\right)^{1-y^{(i)}}$$

→ Log likelihood

$$l(\theta) = \log L(\theta)$$

$$l(\theta) = \sum_{i=1}^{m} y^{(i)} \cdot \log h_\theta(x^{(i)}) + (1-y^{(i)}) \log(1-h_\theta(x^{(i)}))$$

Choose θ to ↑ $l(\theta)$

→ By Ascent — maximise log likelihood

$$\theta_j := \theta_j + \alpha \cdot \frac{\partial}{\partial \theta_j} l(\theta)$$

Climb up the hill, concave fn

$$h_\theta(x) = g(\theta^T x)$$

$$g(z) = \frac{1}{1+e^{-z}}$$

$$l(\theta) = y \log h_\theta(x) + (1-y) \log(1-h_\theta(x))$$

$$\frac{\partial l(\theta)}{\partial \theta_j} = \frac{\partial}{\partial \theta_j} y \log h_\theta(x) + \frac{\partial (1-y) \log(1-h_\theta(x))}{\partial \theta_j}$$

$$\frac{\partial l(\theta)}{\partial \theta_j} = y \cdot \frac{1}{h_\theta(x)} \cdot \frac{\partial h_\theta(x)}{\partial \theta_j} + (1-y) \cdot \frac{1}{(1-h_\theta(x))} \cdot (-1) \cdot \frac{\partial h_\theta(x)}{\partial \theta_j}$$

Now as $h_\theta(x) = g(\theta^T x)$

$$\frac{\partial l(\theta)}{\partial \theta_j} = y \cdot \frac{1}{g(\theta^T x)} \cdot \frac{\partial g(\theta^T x)}{\partial \theta_j} - \frac{(1-y)}{1-g(\theta^T x)} \frac{\partial g(\theta^T x)}{\partial \theta_j}$$

$$\frac{\partial l(\theta)}{\partial \theta_j} = \left[\frac{y}{g(\theta^T x)} - \frac{(1-y)}{(1-g(\theta^T x))} \right] \cdot \frac{\partial g(\theta^T x)}{\partial \theta_j}$$

We Know

$$g'(z) = g(z)\,(1-g(z))$$

$$\frac{\partial g(\theta^T x)}{\partial \theta_j} = g(\theta^T x)\,(1-g(\theta^T x)) \frac{\partial (\theta^T x)}{\partial \theta_j}$$

$$\frac{\partial (\theta^T x)}{\partial \theta_j} = x_j$$

$$\therefore \frac{\partial g(\theta^T x)}{\partial \theta_j} = g(\theta^T x)\,(1-g(\theta^T x)) \cdot x_j$$

$$\frac{\partial l(\theta)}{\partial \theta_j} = \left[\frac{y}{g} - \frac{(1-y)}{(1-g)} \right] \cdot g \cdot (1-g) \cdot x_j$$

$$= \left[y(1-g) - (1-y) \cdot g \right] \cdot x_j$$

$$= \left[y - yg - g + yg \right] x_j$$

$$\frac{\partial l(\theta)}{\partial \theta_j} = \left(y - g(\theta^T x)\right) \cdot x_j$$

$$\frac{\partial l(\theta)}{\partial \theta_j} = \left(y - h_\theta(x)\right) \cdot x_j$$

$$\theta_j := \theta_j + \alpha \sum_{i=1}^{m} \left(y^{(i)} - h_\theta(x^{(i)})\right) \cdot x_j^{(i)}$$

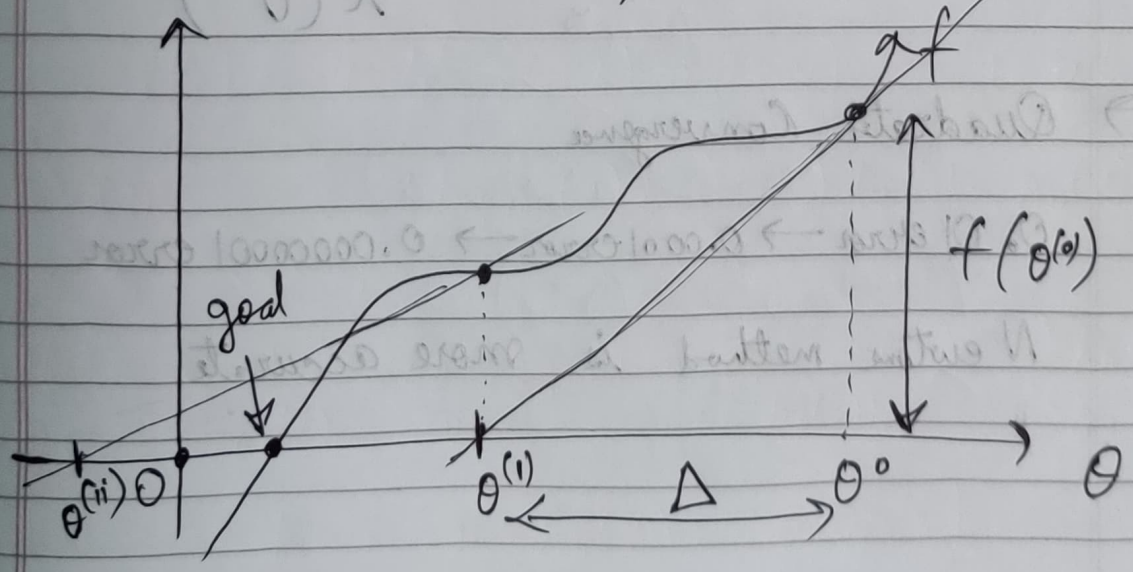
$$\theta := \theta + \alpha \nabla l(\theta)$$

* **Newtons Method** allows to take much bigger iterations to get value of θ.

Have a fn f | Very fast algorithm.

We want find θ st $f(\theta) = 0$

want to maximize $l(\theta)$ i.e. want $l'(\theta) = 0$



$$\theta^{(1)} := \theta^{(0)} - \Delta$$

$$f'(\theta^{(0)}) = \frac{f(\theta^{(0)})}{\Delta}$$

$$\Delta = \frac{f(\theta^{(0)})}{f'(\theta^{(0)})}$$

$$\theta^{(t+1)} := \theta^{(t)} - \frac{f(\theta^{(t)})}{f'(\theta^{(t)})}$$

Let $f(\theta) = l'(\theta)$

$$\theta^{(t+1)} := \theta^{(t)} - \frac{l'(\theta^{(t)})}{l''(\theta^{(t)})}$$

→ Quadratic Convergence

0.01 error → 0.0001 error → 0.00000001 error

Newtons method is more accurate

When θ is a vector

$$\theta^{(t+1)} := \theta^{(t)} - H^{-1} \underbrace{\nabla_\theta l}_{\text{vector } \mathbb{R}^{n+1}}$$

$\theta \in \mathbb{R}^{n+1}$

$H \rightarrow$ Hessian matrix $\in \mathbb{R}^{n+1 \times n+1}$

$H_{ij} = \dfrac{\partial^2 l}{\partial \theta_i \partial \theta_j}$ $\rightarrow$ for n parameters

$$H = \begin{bmatrix} \dfrac{\partial^2 f}{\partial \theta_1^2} & \dfrac{\partial^2 f}{\partial \theta_1 \partial \theta_2} \\ \dfrac{\partial^2 f}{\partial \theta_2 \partial \theta_1} & \dfrac{\partial^2 f}{\partial \theta_2^2} \end{bmatrix}$$

PSO

Id $f(x) = g(a^T x)$

$g: \mathbb{R} \to \mathbb{R}$

$a \in \mathbb{R}^n$ is a vector

$\nabla f(x) = ?$, $\nabla^2 f(x) = ?$

$f(x) = g(a^T x)$

$\nabla f(x) = g'(a^T x) \cdot \frac{\partial}{\partial x}(a^T x)$

$= g'(a^T x) \cdot a$

$\nabla f(x) = g'(a^T x) \cdot a$

$\nabla^2 f(x) = \frac{\partial}{\partial x} g'(a^T x) \cdot a$

$= a \cdot g''(a^T x) \cdot a$

$\nabla^2 f(x) = a\, g''(a^T x)\, a^T$

6 June 2026 — Saturday — 11:40 AM

|| ॐ नमो भगवते वासुदेवाय नमः ||

* Norms

A norm of a vector $||x||$ is informally a measure of the "length" of the vector.

$$||x||_2 = \sqrt{\sum_{i=1}^{n} x_i^2}$$

$$||x||_2^2 = x^T x$$

A norm is a fn $f : \mathbb{R}^n \longrightarrow \mathbb{R}$ that has 4 props:

1) $\forall\, x \in \mathbb{R}^n,\ f(x) \geq 0$ (non-ve)

2) $f(x) = 0$ iff $x = 0$ (definiteness)

3) $\forall\, x \in \mathbb{R}^n,\ t \in \mathbb{R},\ f(tx) = |t| f(x)$ homogeneity.

4) $\forall\, x, y \in \mathbb{R}^n,\ f(x+y) \leq f(x) + f(y)$ (Δ inequality)

l_1 norm, $||x||_1 = \sum_{i=1}^{n} |x_i|$

l_∞ norm, $||x||_\infty = \max_i |x_i|$

family of norms, l_p norm $\quad p \geq 1 \quad ||x||_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p}$

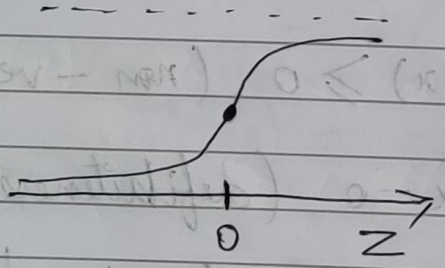
Norms for matrices, such as Frobenius Norm

$$\|A\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} A_{ij}^2} = \sqrt{tr(A^T A)}$$

* Lecture - 4 — Perceptron & Generalized Linear Models

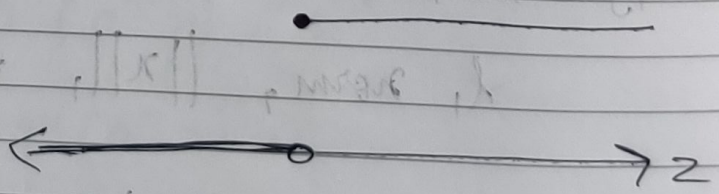
- Perceptron family
- GLM
- Exponential Family
- Softmax Regression — Multi Class classification

→ Logistic Regression uses Sigmoid fn



$$g(z) = \frac{1}{1+e^{-z}}$$

- Softmax fn

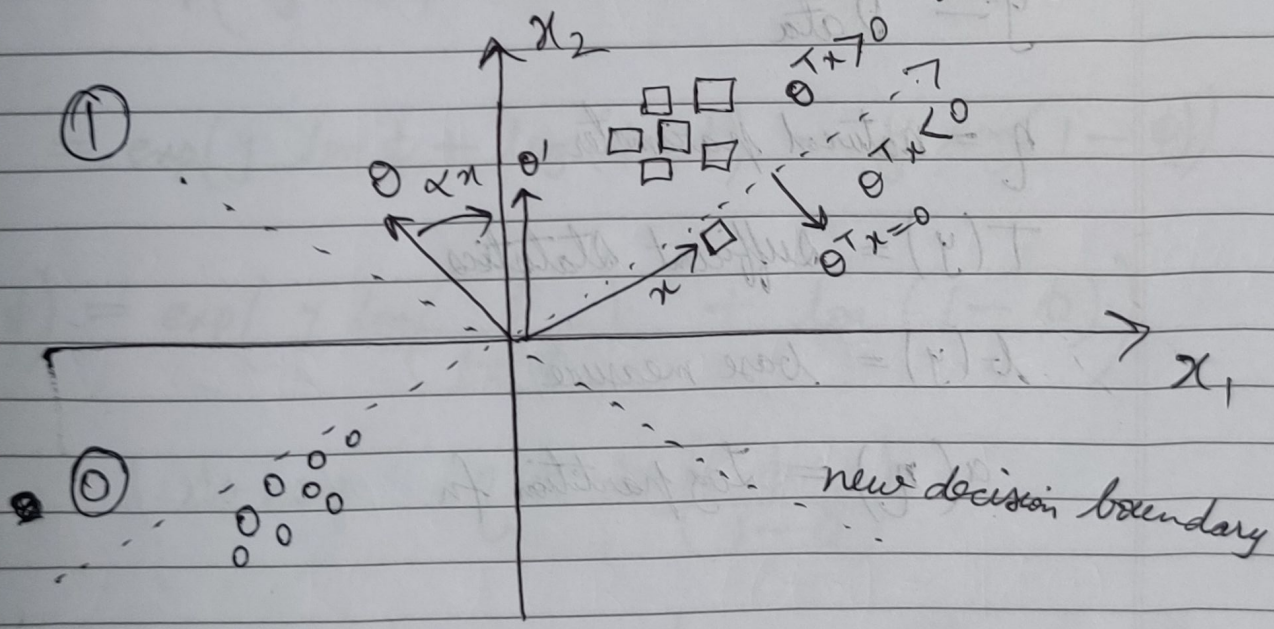


$$g(z) = \begin{cases} 1 & ; z \geq 0 \\ 0 & ; z < 0 \end{cases}$$

$$h_\theta(x) = g(\theta^T x)$$

$$\theta_j := \theta_j + \alpha\left(y^{(i)} - h_\theta(x^{(i)})\right) x_j^{(i)}$$

Scalar $0 \rightarrow$ algorithm got it right
$| \rightarrow$ if wrong $y^{(i)} = 1$
$-| \rightarrow$ if wrong $y^{(i)} = 0$



$$\theta \approx x \mid y = 1$$

$$\theta \not\approx x \mid y = 0$$

* Exponential Family

Class of probability distributions fn

PDF (Probability Density fn)

$$P(y;\eta) = b(y)\ \exp\left(\eta^T T(y) - a(\eta)\right)$$

y = Data

η = natural parameter

$T(y)$ = Sufficient statistics

$b(y)$ = base measure

$a(\eta)$ = Log partition fn

* Bernoulli (Binary Data)

ϕ = probability of event happening or not

$$P(y;\phi) = \phi^y (1-\phi)^{(1-y)}$$

$$= \exp\left(\log\left(\phi^y \cdot (1-\phi)^{(1-y)}\right)\right)$$

$$= \exp\left(y \log\phi + (1-y)\log(1-\phi)\right)$$

$$= \exp\left(y \log\phi + \log(1-\phi) - y\log(1-\phi)\right)$$

$$P(y;\phi) = \exp\left(y \log\left(\frac{\phi}{1-\phi}\right) + \log(1-\phi)\right)$$

So here $\eta = \log\left(\frac{\phi}{1-\phi}\right)$

$$\phi = \frac{1}{1+e^{-\eta}}$$

$$T(y) = y$$

$$a(\eta) = -\log(1-\phi)$$

$$= \log(1+e^{\eta})$$

$$b(y) = 1$$

* Gaussian (with fixed variance)

Assuming $\sigma^2 = 1$

$$P(y;\mu) = \frac{1}{\sqrt{2\pi}} \exp\left(\frac{-1}{2}(y-\mu)^2\right)$$ Gaussian PDF

$$P(y;\mu) = \frac{1}{\sqrt{2\pi}} \exp\left(-\frac{1}{2}y^2\right) \exp\left(\mu y - \frac{1}{2}\mu^2\right)$$

$$\eta = \mu$$

$$T(y) = y$$

$$a(\eta) = \mu^2/2 = \eta^2/2$$

$$b(y) = \frac{1}{\sqrt{2\pi}} \exp(-y^2/2)$$

* Properties

(a) MLE wrt $\eta \Rightarrow$ Concave

NLL (-ve log likelihood) is convex

(b) $$E[y;\eta] = \frac{\partial (a(\eta))}{\partial \eta}$$

(c) $$Var[y;\eta] = \frac{\partial^2 a(\eta)}{\partial \eta^2}$$

Proof: For a continous random variable:

$$P(y;\eta) = b(y)\exp(\eta y - a(\eta))$$

Density must $\int$ to 1

$$\int P(y;\eta) = 1$$

$$\int b(y)\, e^{\eta y - a(\eta)}\, dy = 1$$

$$e^{-a(\eta)} \int b(y)\cdot e^{\eta y}\, dy = 1$$

$$\int b(y)\cdot e^{\eta y}\, dy = e^{a(\eta)} \longrightarrow (1)$$

Differentiate both sides

$$\frac{d}{d\eta} e^{a(\eta)} = \frac{d}{d\eta}\int b(y)\, e^{\eta y}\, dy$$

$$e^{a(\eta)}\cdot a'(\eta) = \int y\, b(y)\, e^{\eta y}\, dy$$

$$a'(\eta) = \frac{\int y\, b(y)\, e^{\eta y}\, dy}{\int b(y)\cdot e^{\eta y}\, dy}$$

using (1)

Now $P(y;\eta) = \dfrac{b(y)\, e^{\eta y}}{\int b(y)\, e^{\eta y}\, dy}$

$\therefore a(\eta) = \int y \cdot P(y;\eta)\, dy$

$E[Y] = \int y \cdot P(y;\eta)\, dy$

$\therefore E[Y] = a'(\eta)$

Tells average prediction

Proof 2 Differentiate again

$a'(\eta) = \dfrac{N}{D}$

here, $N = \int y\, b(y)\, e^{\eta y}\, dy$

$D = \int b(y)\, e^{\eta y}\, dy$

$a''(\eta) = \dfrac{N'D - D'N}{D^2}$

(y is const & η is parameter)

$N' = \int y^2\, b(y)\, e^{\eta y}\, dy$

$D' = \int y\, b(y)\, e^{\eta y}\, dy = N$

$$\frac{d}{d\eta}\int f(y,\eta)\,dy = \int \frac{\partial}{\partial \eta} f(y,\eta)\,dy$$

$$D(\eta) = \int b(y)\, e^{\eta y}\, dy$$

∴ The normalization constant contains all moments

$E[Y]$, $E[Y^2]$, $E[Y^3]$ and so on

$$\therefore \quad a(\eta) = \log D(\eta)$$

is called moment generating object

Its derivates reveal the statistics of the distribution

→ Fisher Information $I(\eta) = a''(\eta)$

- Natural Gradient Descent
- Reinforcement Learning
- Large scale optimization

→ Variational Inference : The ELBO (Evidence Lower Bound) gradients depend on derivates of $a(\eta)$

$$\text{Now, } a''(\eta) = \frac{\left(\int y^2\, b(y) e^{\eta y} dy\right) D - N^2}{D^2}$$

$$a''(\eta) = E[Y^2] - (E[Y])^2$$

$$a''(\eta) = Var(Y)$$

Tells us uncertainity

→ Fisher Information

$$I(\eta) = E\left[\left(\frac{\partial \log P_\theta(y;\eta)}{\partial \eta}\right)^2\right]$$

For exponential Families

$$I(\eta) = a''(\eta) = Var(Y)$$

Heavily used in

- Natural Gradient Descent
- Information Geometry
- RL
- LLM optimization

→ VAEs (Variational Auto Encoders)

Assume Gaussian latent variables

$$Z \sim N(\mu, \sigma^2)$$

KL divergence formulas become traceable because Gaussian is an exponential family distribution.

The gradients of the ELBO involve derivatives of log partition fn

→ The log-partition fn $a(\eta)$ is much more than a normalization constant.

It is generator of moments:

$$a'(\eta) = E[Y]$$

$$a''(\eta) = Var(Y)$$

In higher dimensions

$$\nabla a(\eta) = E[T(Y)]$$

$$\nabla^2 a(\eta) = Cov(T(Y))$$

✳ GLM — Generalized Linear Models

Assumptions / Design choices

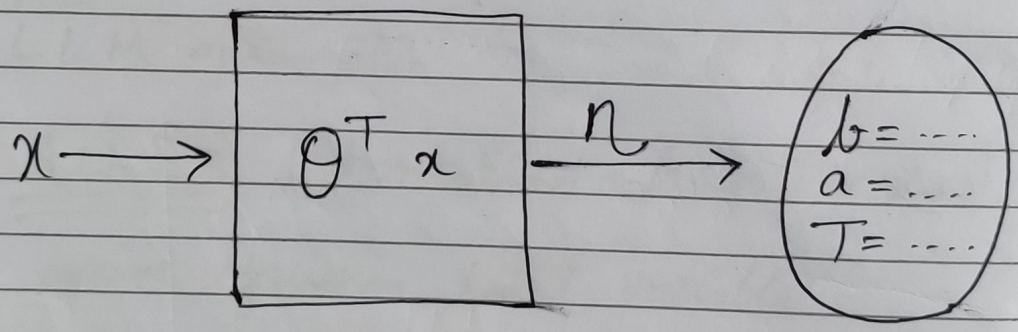
(i) $y|x;\theta \sim$ Exponential family(η)

(ii) $\eta = \theta^T x$ $\quad \theta \in \mathbb{R}^n, x \in \mathbb{R}^n$

(iii) Test time : output $E[y|x;\theta]$

$h_\theta(x) = E[y|x;\theta]$

Linear Model $\qquad$ Exponential Family



$h_\theta(x) = E[y;\eta] = E[y;\theta^T x]$

→ During Training → $\max_\theta \log\left(P\left(y^{(i)};\theta^T x^{(i)}\right)\right)$

✳ GLM Training

Learning update rule ⟹ $\theta_j := \theta_j + \alpha\left(y^{(i)} - h_\theta(x^{(i)})\right)x_j^{(i)}$

२।९।।

classmate
Date
Page 60

Q) What probability distribution should I use for my target variable y?

Problem	Target Variable	Distribution
• House price prediction	Continuous	Gaussian
• Spam detection	0 or 1	Bernoulli
• Multi-Class classification	one of K classes	Multinomial
• No. of customers entering store	Count	Poisson
• Time until machine failure	+ve continuous	Exponential / Gamma

- η = natural parameter

$\mu = E[y; \eta] = g(\eta) \longrightarrow$ Canonical Response fn

$\eta = g^{-1}(\mu) \longrightarrow$ Canonical link fn

$g = g(\eta) = \frac{\partial a(\eta)}{\partial \eta}$

- θ = Model Parameters.
- Canonical Parameters — ϕ — Bernoulli; μ, σ^2 — Gaussian; λ — Poisson

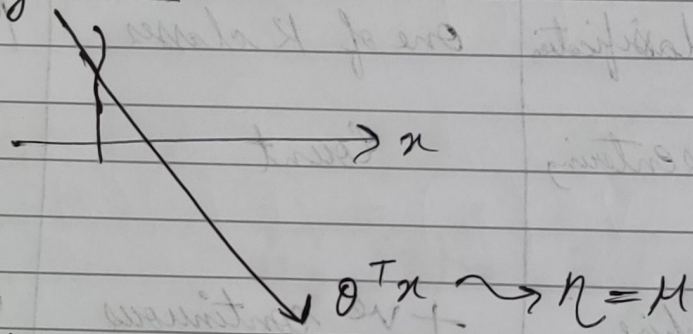
→ Logistic Regression

$$h_\theta(x) = E[y|x;\theta] = \phi = \frac{1}{1+e^{-\eta}} = \frac{1}{1+e^{-\theta^T x}}$$

Bernoulli Distribution

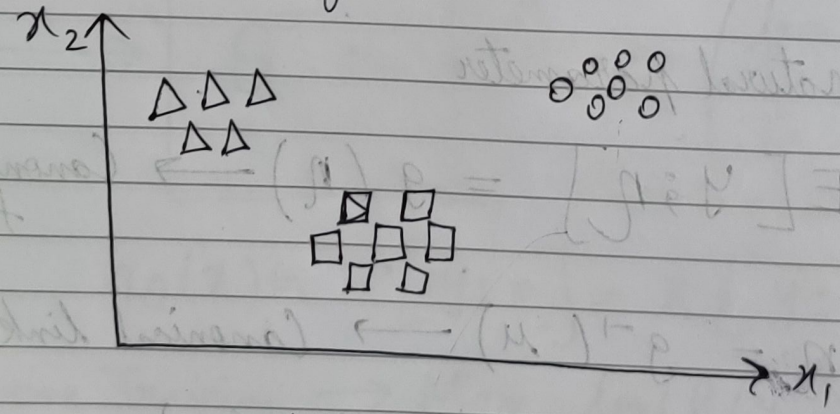
→ Assumptions

1) Regression



20 June Sunday

⟹ Softmax Regression ← Cross Entropy



K = no. of classes

$x^{(i)} \in \mathbb{R}^n$

one hot vector Label $y = [\{0,1\}^K]$ eg $[0,0,1,0]$

॥ ॐ नमो भगवते वासुदेवाय नमः ॥

18 June 2026 — Thursday — 6:58 PM

→ PSD matrices are non-negatives

PSD Matrix is defined by: $x^T A x \geq 0$ for every vector x.

- Squared Length $x^T x = \|x\|^2$
- Covariance $Var(X) = E[(X - \mu)^2]$
- Linear Regression Hessian $H = X^T X$

$$z^T H z = z^T X^T X z = (Xz)^T (Xz) = \|Xz\|^2$$

$\theta_{class} \in \mathbb{R}^N$ K such class $\in \{\Delta, \circ, \square\}$

$$K \begin{bmatrix} \overset{n}{} \\ — \theta_c — \\ — \theta_c — \\ \end{bmatrix}$$